# Registro de Decisiones, Riesgos y Limitaciones

**Decisiones del equipo IOND sobre el flujo SDLC y el rol de QA como Analista de Calidad y Requerimientos**

Propuesta por: Steve Nina — Scrum Master IOND
Aplica a: IOND · IonMind · StageMind
Audiencia: PO · Lead Dev / RM · Lead QA · Scrum Master · Development · QA

## Cómo usar este documento

Este documento registra las **55 decisiones** (Q-01 a Q-55) que consolidan el flujo SDLC de IOND: Q-01 a Q-46, tomadas por el equipo el **2026-10-09**; Q-47 a Q-54 (estrategia de pruebas del release y herramientas de QA), del **2026-10-10**, y Q-55 (asignación del QA), del **2026-10-11**. Cada decisión trae el escenario de negocio que la motivó, la decisión adoptada y los documentos donde se aplica.

- **Q-xx:** decisiones (sección 2).
- **AV-xx:** acciones de verificación. No son decisiones pendientes, sino tareas de configuración o de inventario con responsable (sección 3).
- **O-xx:** observaciones de capacidad y organización (sección 4).
- **F-xx:** inconsistencias detectadas en la guía original y cómo quedaron resueltas (sección 5).
- **R-xx:** riesgos con probabilidad, impacto y mitigación (sección 6).
- **L-xx:** limitaciones de las herramientas y del entorno (sección 7).
- **Registro de decisiones** en una tabla (sección 8).

La fuente canónica que aplica estas decisiones es `knowledge/L1-project/sdlc-consolidation.md` (v2.3).

---

## 1. Resumen de las decisiones clave

1. **QA como Analista de Calidad y Requerimientos:** QA redacta la Matriz de Requerimientos, normalizando y completando la especificación del PO (no la reescribe). **QA propone y el PO decide**; el PO da su VoBo con un único comentario.
2. **Un PO para los tres productos** (IOND, IonMind y StageMind). Los solicitantes externos participan como Stakeholders.
3. **El LD es también el RM.** Sus PRs los revisan dos miembros del equipo con mayor antigüedad y **ambos deben aprobar**. Además, el LD despliega cada ticket en el ambiente de QA con los workflows de DevOps.
4. **QA rechaza solo contra la matriz** y el contrato aprobado en `prototype review`. Lo demás se registra como ticket nuevo ( `fuera-de-matriz` ).
5. **Discovery Review semanal** en un día dedicado, acordado con el SH. El SH puede participar por video cuando no pueda asistir.
6. **Release semanal en 5 días:** corte el lunes, Go/No-Go el viernes con quórum PO + LQA + RM. No hay releases vacíos y los PRs se integran con merge atómico.
7. **Fix-forward para los bloqueantes del release:** decisión en ≤ 4 h y fix en ≤ 1 día; si no se logra, el ticket se revierte ( `diferido` ) o se declara NO-GO. Hay además un carril de hotfix para incidentes en producción.
8. **Tipos de estado en ClickUp:** se mantienen los actuales ( `uat` , `ready to merge` y `merged` en Done; `merged` es el cierre; `invalid` en Closed). En Discovery, `to do` pasa a Active.
9. **Improvements no tiene UAT.** En un caso excepcional decide el PO.
10. **Campo nuevo Stagemind.Version.** Los tickets de Gateway siguen en el SDLC y usan el campo heredado G.Version.
11. **Cuatro suites en el release:** un **smoke maestro único** para todo release (preprod y producción con una company de prueba), un **sanity** y una **regresión** propios de cada versión, y el **E2E maestro** con todas las features como segunda capa de `regression` .
12. **Las subtareas en `done w/d` siguen al padre** a `ready to merge` y a `merged` .
13. **El catalyst prepara y QA publica:** las herramientas leen ClickUp y bot-test sin modificarlos y generan matrices, comentarios y comandos de ejecución.

14. **El LQA asigna al QA desde `assignment`:** el mismo QA sigue el R&D y el prototipo, redacta la matriz en `requirements` y ejecuta `mock testing`.

---

## 2. Decisiones (Q-01 a Q-55)

### 2.1 Roles y gobierno

#### Q-01 · ¿Quién es el PO de cada producto y quién da el VoBo cuando el solicitante es otro equipo? ✅

- **Escenario:** un equipo externo pide que StageMind devuelva nombre y descripción de producto en una sola llamada.
- **Decisión: un PO por producto; en IOND es la misma persona para los tres productos (Marcel Herrera Rendón).** El VoBo siempre lo da el PO. El solicitante externo participa como SH en `prototype review` y en `uat`.
- **Se aplica en:** 04 §3 y anexo; 05 Anexo C; fuente canónica §2.

#### Q-02 · ¿Quién es el Lead Dev y qué pasa si coincide con el RM? ✅

- **Escenario:** si el RM también es LD y desarrolla un ticket, nadie revisaría su código con independencia.
- **Decisión: el LD es el mismo que el RM (Rodolfo Merlo Ali).** Sus PRs los revisan **dos miembros del equipo con mayor antigüedad y ambos deben aprobar** antes del merge.
- **Se aplica en:** 04 §3; fuente canónica §2 y D-09; R-08.

#### Q-03 · ¿Quién facilita cuando el SM es parte de un desacuerdo? ✅

- **Escenario:** QA rechaza un ticket, el Dev no está de acuerdo, y el SM es quien rechazó.
- **Decisión: facilita el LD/RM** (opción B). La decisión sobre los requerimientos sigue siendo del PO. **Soporte no es un rol del equipo IOND:** se trabaja con el servicio de soporte general de la empresa.
- **Se aplica en:** 04 §7; 06a §11; fuente canónica §8.4.

#### Q-04 · ¿Quién despliega el build del ticket en el ambiente de QA? ✅

- **Escenario:** un ticket de billing requiere migraciones en los tenants y variables nuevas de la pasarela de pago.
- **Decisión: lo despliega el LD**, con los workflows preparados por DevOps. El handoff del Dev (C-09) sigue documentando migraciones, variables y datos.
- **Se aplica en:** fuente canónica D-10; 06b.

#### Q-05 · ¿Dónde se hace `mock testing`? ✅

- **Escenario:** se integra el tracking de un carrier que no ofrece sandbox.
- **Decisión:** en el **ambiente asignado al QA**, con la rama del ticket. Si una integración no tiene sandbox, la sección *Datos de prueba* de la matriz indica cómo se prueba: mock, colección simulada o cuenta de prueba.
- **Se aplica en:** fuente canónica D-11; 05 §4.9; 06b.

#### Q-06 · ¿Los Stakeholders pueden participar de forma asíncrona? ✅

- **Escenario:** un cliente externo no puede asistir al Pilot de una integración que solicitó.
- **Decisión:** hasta ahora el SH siempre pudo asistir. Si no puede, **participa de forma asíncrona por video** (demo grabada y comentario en el ticket en ≤ 2 días hábiles). El PO define por ticket quién es SH obligatorio.
- **Se aplica en:** 04 §3; fuente canónica §2.

### Q-07 · ¿Reuniones por ticket o una sesión semanal? ✅

- **Escenario:** Request, Prototype Review y Pilot son reuniones obligatorias, y convocarlas una por una satura la agenda.
- **Decisión: Discovery Review semanal, en un día dedicado.** El día se acuerda con el SH (AV-01). La agenda va por bloques: requests nuevos → prototipos → pilots. Los tickets pequeños se tratan de forma asíncrona.
- **Se aplica en:** 04 §4; 06a §9; fuente canónica D-01.

## 2.2 Discovery

### Q-08 · ¿Todo ticket de Discovery pasa por R&D y `prototype review`? ✅

- **Escenario:** agregar un nodo simple frente a hacer configurable el motor de persistencia.
- **Decisión:** sí, con timebox de R&D por tamaño: **S ≤ 1 día · M ≤ 3 días · L ≤ 5 días**. `prototype review` es siempre obligatorio porque ahí nace la línea base.

### Q-09 · ¿Qué es un "prototipo" según el tipo de ticket? ✅

- **Decisión:**

| Tipo de ticket | Prototipo esperado |
|---|---|
| UI / UX | Wireframe o mock navegable |
| API / backend | Contrato OpenAPI: endpoints, request/response, códigos de error, límites de parámetros |
| Integración (conectores, carriers, marketplaces) | Diseño del board o flow, mapeo de datos y manejo de errores |
| IA (IonMind / StageMind) | Dataset de ejemplo, salida esperada y criterio de calidad medible |
| Refactor / técnico | Diagrama de arquitectura actual vs propuesta y requisitos no funcionales que deben preservarse |

### Q-10 · ¿El contrato aprobado en `prototype review` es línea base de rechazo? ✅

- **Escenario:** el contrato fija una longitud del título de 10–200 caracteres y un `400` para imagen inaccesible. En `mock testing` el título sale con 469 caracteres y el endpoint devuelve `422`.
- **Decisión: sí, cuando la matriz lo referencia** (`REQ-xx: cumplir el contrato vN`). Lo que no esté ni en la matriz ni en el contrato es un hallazgo fuera de matriz.

### Q-11 · ¿Qué se hace con los ajustes del `pilot presentation`? ✅

- **Escenario:** en el Pilot de un botón que genera un mapeo automático, el PO pide una vista previa antes de aplicarlo.
- **Decisión:** cada ajuste se clasifica en el acta C-10:
  - **Contradice la matriz:** se corrige en el ticket.
  - **Es nuevo e imprescindible** para el valor mínimo: cambio de alcance (matriz v+1 + nuevo VoBo).
  - **Es nuevo y no imprescindible:** ticket en Improvements con `pilot-feedback`.

### Q-12 · Épicas con subtareas: ¿una matriz o varias? ✅

- **Decisión: una sola matriz en el ticket padre**, con los REQ agrupados por subtarea (`REQ-PAT-01`, `REQ-BRD-01`…). QA prueba el padre. Las subtareas cierran en `done w/d` sin QA individual.

### Q-13 · ¿Cómo se expresan los requerimientos de refactors y POCs técnicos? ✅

- **Decisión:** con **REQ de invariancia** (el comportamiento observable no cambia) más **REQ-NF medibles** (latencia p95, throughput, recuperación ante fallas) y regresión focalizada.

### Q-14 · ¿Cómo cierran los spikes de investigación? ✅

- **Decisión: opción A.** Los spikes se crean como **subtareas** de la funcionalidad y cierran en `done w/d`, con el documento enlazado. Si no hay funcionalidad padre, el PO decide si se crea un ticket padre de investigación.

### Q-15 · ¿Quién clasifica un ticket (tablero, TYPE)? ✅

- **Escenario:** "integrar un widget de terceros para feedback dentro de la app" es TYPE New Feature, pero no cambia reglas de negocio.
- **Decisión:**
  - **En Discovery clasifica el PO.**
  - **En los bugs, QA clasifica según su criterio** (tablero, TYPE y prioridad = severidad), **con la aprobación del PO** en el triage (C-33).
  - El tablero se decide por el impacto y el TYPE por la naturaleza del ticket. Una New Feature pequeña puede ir a Improvements, con `requirements` obligatorio.

### Q-16 · UAT: ¿qué tickets la requieren, cuándo y dónde? ✅

- **Decisión:** el PO marca `uat-requerido` en `requirements`. La UAT ocurre **antes** de `ready to merge`, en el ambiente de QA o preprod con la rama del ticket. El flujo queda `mock testing` → `uat` → `ready to merge`.

### Q-17 · ¿Qué pasa si el VoBo del PO no llega en 24 h? ✅

- **Decisión: no hay aprobación tácita.**
  - A las 24 h, QA vuelve a mencionar al PO y el SM lo plantea en la daily.
  - A las 48 h, el ticket pasa a `blocked` con `bloq-definicion`.
  - Mientras tanto, QA prepara la Matriz de Pruebas y los datos.

### Q-18 · ¿Quién puede pedir cambios de alcance durante el desarrollo? ✅

- **Escenario:** con el checkout del marketplace en desarrollo, negocio pide aceptar una segunda moneda.
- **Decisión: solo a través del PO.** QA actualiza la matriz a v+1 (C-08, `cambio-alcance`), el PO vuelve a dar el VoBo y **después** se modifica el código.

## 2.3 QA como Analista de Calidad y Requerimientos

### Q-19 · ¿QA reescribe la especificación del PO en la matriz? ✅

- **Decisión: opción B.** QA **normaliza** la especificación en REQ-IDs referenciando el texto del PO, completa condiciones, excepciones y límites, y agrega solo lo que falta.

### Q-20 · ¿Hasta dónde puede QA proponer reglas de negocio? ✅

- **Escenario:** al redactar la matriz de "cambio de plan", QA nota que no está definido si los consumos del periodo se reinician en un upgrade.
- **Decisión: QA propone** una opción recomendada (C-05, "Puntos que requieren tu decisión") y **el PO decide**. Ninguna regla queda "a criterio del Dev".

### Q-21 · ¿Por qué canal y con qué plazo se resuelven las reglas de negocio no definidas? ✅

- **Decisión:** el canal es **la matriz más C-05**, con plazo de **VoBo en 24 h**. Si una regla no se resuelve, solo ese REQ queda bloqueado y el resto avanza.
- **Reglas típicas que deben resolverse en la matriz antes de desarrollar:**
  - Ejecución de gracia al agotar el saldo.
  - Reinicio de consumos y proration al cambiar de plan.
  - Cuotas por plan.
  - Moneda por comprador × vendedor y carritos mixtos.
  - Factura por cada clic en "Suscribirse".
  - Cancelación inmediata o al final del periodo.
  - Storefront sin base plan.
  - Eliminar un service con subscribers activos.
  - Retirar un método de pago con suscripciones activas.
  - Comportamiento exacto del cambio de plan.

### Q-22 · ¿Cómo se gestionan los hallazgos fuera de matriz? ✅

- **Decisión: no son causa de rechazo.** QA crea un ticket con `fuera-de-matriz` (C-17) y el PO lo prioriza. Si el PO considera que sin ese cambio el ticket no tiene valor, aplica un cambio de alcance (C-08).

### Q-23 · ¿El Dev debe acusar lectura de la matriz? ✅

- **Decisión:** sí, mediante un **ítem de checklist nativo** en el ticket: "*Matriz de Requerimientos vN leída por el Dev*". Lo marca el Dev antes de pasar a `to do` y va en las plantillas T-01 y T-03. Las dudas se comentan con C-07.

### Q-24 · ¿Quién define los umbrales de calidad para salidas de IA? ✅

- **Escenario:** "normalizar logos a formato cuadrado 1:1 sin deformar ni recortar".
- **Decisión:** el **PO** define el umbral (p. ej. ≥ 95 % de un corpus de N logos). **QA** propone el dataset y los casos borde. Las exclusiones van a "Fuera de alcance".

### Q-25 · ¿Una sola escala de severidad? ✅

- **Decisión:** sí. Se usa la prioridad nativa: `urgent` (bloqueante), `high`, `normal` y `low`, con las definiciones de la fuente canónica §10. `urgent` siempre bloquea.

### Q-26 · ¿Los QA Points entran en la planificación de capacidad? ✅

- **Decisión:** sí. QA los estima al salir de `requirements`. El LQA balancea la carga por Reviewer QA y limita el lote del release con ellos.

### Q-27 · ¿Cuándo un requerimiento se separa en otro ticket? ✅

- **Escenario:** durante la definición de servicios del marketplace surge "habilitar GRAPPs para companies".
- **Decisión:** si agrega un actor, una regla de negocio o un módulo nuevo, es **otro ticket**. Lo decide el PO y queda en "Fuera de alcance".

## 2.4 Improvements

### Q-28 · ¿Quién crea bugs, por qué canal y quién hace el triage? ✅

- **Decisión:** cualquiera los crea (QA, Dev, PO o el soporte general de la empresa) mediante el **Form de intake** (T-02). Hay un **triage semanal** con PO + QA (y el LD si hace falta).

### Q-29 · ¿Cuándo una mejora requiere `requirements`? ✅

- **Decisión:** cuando el criterio no cabe en ≤ 3 líneas verificables, cuando toca validaciones o mensajes al usuario, o cuando el TYPE es New Feature. Los bugs nunca pasan por `requirements`.

### Q-30 · ¿Improvements necesita UAT? ✅

- **Decisión: no hay estado `uat` en Improvements.** En un caso muy excepcional que requiera validación del cliente, **el PO decide** si se valida en el `sanity` del release o si el ticket pasa a Discovery.

### Q-31 · ¿Causas de bloqueo y escalamiento? ✅

- **Decisión:**
  - **Tags de causa:** `bloq-externo`, `bloq-ambiente`, `bloq-definicion`, `bloq-dependencia`.
  - **48 h:** escalamiento automático.
  - **72 h:** el PO, el LD y el SM deciden entre reprogramar, dividir el ticket o (solo si la causa es de alcance) reclasificarlo a Discovery.

## 2.5 Releases

### Q-32 · Corte, calendario, release vacío y límite de lote ✅

- **Decisión:**
  - **Corte:** lunes 09:00.
  - **Calendario:** `dev preparation` y `unit test & ci` el lunes AM · `smoke` del lunes PM al martes AM · `sanity` martes y miércoles · `regression` jueves · **Go/No-Go viernes 10:00**.
  - **Sin tickets no hay release.**
  - **Límite de lote** por capacidad de QA (≤ 15 QA Points de sanity por QA y semana).
- **Siguiente paso acordado:** planificar la matriz de smoke, las features automatizables, `sanity` y `regression`. **Resuelto en Q-47 a Q-51** (sección 2.7).

### Q-33 · Ramas RC por repositorio y merge atómico ✅

- **Decisión: merge atómico.** Todos los PRs de un ticket entran a la RC juntos o ninguno. El RM documenta las ramas RC por repositorio en C-23.

### Q-34 · Fix-forward ✅

- **Decisión:**
  - Los bugs bloqueantes del release se corrigen en la rama RC (**fix-forward**). La decisión la toman el PO, el LD y el RM en ≤ 4 h, y el fix debe estar en ≤ 1 día hábil. Si no se logra, el ticket se revierte de la RC ( `diferido` ) o se declara NO-GO.
  - El resto de bugs pasa a Improvements, previa discusión con el PO y el LD.
  - El protocolo paso a paso está en la guía 06b (Development + QA).

### Q-35 · Go/No-Go: quórum y CONDITIONAL GO ✅

- **Decisión:**
  - **Quórum:** PO + LQA + RM.
  - **CONDITIONAL GO:** solo con hallazgos `high` que tengan workaround, riesgo aceptado por el PO y fix agendado. Un `urgent` abierto es NO-GO.
  - **Ante un NO-GO**, la tarea de Release permanece en `regression` con un C-29 NO-GO y se reprograma su fecha de vencimiento al siguiente Go/No-Go. Si se desarma la versión, se cierra con un comentario y los tickets revertidos llevan `diferido`.

### Q-36 · Despliegue con DevOps externo ✅

- **Decisión:**
  - Ventana fija acordada con DevOps: viernes antes de las 14:00 o lunes temprano (AV-07).
  - Rollback documentado en la tarea de Release y probado en preprod.
  - El PO comunica a los clientes con la release note de cliente, apoyado por el soporte general de la empresa.

### Q-37 · Releases de varios productos en la misma semana ✅

- **Decisión: una tarea de Release por producto y versión**, relacionadas entre sí. El orden de despliegue sigue las dependencias (StageMind → IonMind → iond).

### Q-38 · Carril de hotfix ✅

- **Decisión:** el incidente se reporta (soporte general de la empresa, clientes o monitoreo) y se crea un ticket en Improvements con prioridad `urgent` y tags `hotfix` + `det-prod`. Pasa directo a `to do` y sigue el flujo normal hasta `ready to merge`. Luego sale en un release extraordinario con smoke y sanity focalizados. Autorizan el PO y el LD; ejecuta el RM. **Objetivo: ≤ 24 h a producción.**

### Q-39 · Validación continua ✅

- **Decisión:**
  - **Bug bash** quincenal de 60–90 min (QA + Dev + SM). Los hallazgos van a `backlog` con `det-exploratorio`.
  - **Post-despliegue:** smoke en producción en ≤ 2 h (QA) y monitoreo de 48 h (LD/RM con DevOps).

### Q-40 · Alcance de `regression` y automatización ✅

- **Decisión:** regresión **basada en riesgo**: módulos impactados, baseline crítico y flujos cross-system. La automatización se **prioriza por etapa** mientras se completan todos los flujos E2E:

| Etapa | ¿Automatizar? | Cómo |
|---|---|---|
| `unit test & ci` | Ya automatizada | Pipeline de CI mantenido por Development |
| `smoke` | **Sí, al 100 % (prioridad 1)** | E2E con el proyecto Playwright existente + checks de salud de las APIs (iond, IonMind, StageMind); se reutiliza en producción |
| `regression` | **Sí, de forma progresiva (prioridad 2)** | Primero pruebas de API y de contrato entre productos; luego E2E del baseline crítico (meta orientativa: 60–70 % del baseline al cierre del piloto F4) |
| `sanity` | **En parte** | Reutiliza las pruebas de API del Dev; los TC que se estabilizan pasan a la regresión automatizada |
| Salidas de IA | Asistida | Scripts sobre el corpus más revisión humana de los casos dudosos |

- **Complemento:** la composición de cada suite y la hoja de ruta de automatización por olas se definen en Q-47 a Q-51.

## 2.6 ClickUp y datos

### Q-41 · Orden y tipos de estado ✅

- **Decisión:**
  - **Se mantienen los tipos actuales:** `uat`, `ready to merge` y `merged` en **Done**; `merged` es el último estado y el cierre del flujo; `invalid` en **Closed**.
  - **En Discovery,** `to do` **pasa a Active** y se ubica después de `requirements`.
  - `done w/d` pasa a Done (cierre de subtareas).
  - `blocked` es Active.
  - **Efecto en métricas:** las tarjetas nativas de lead time y cycle time se detienen en el primer Done (`uat` o `ready to merge`). El tiempo hasta `merged` se mide con Time in Status.
- **Se aplica en:** fuente canónica §4; 06a §2; 07.

### Q-42 · Versión de StageMind ✅

- **Decisión:** el SM crea el custom field **Stagemind.Version** (`develop` / vX.Y.Z), y el tag `stagemind-vX.Y.Z` queda retirado.

### Q-43 · Valores de TYPE ✅

- **Decisión:** `Bug`, `New Feature`, `Improvement`, `Refactor` y `Task`, con las definiciones de la fuente canónica §3.2.

### Q-44 · Tickets con Project = Gateway ✅

- **Decisión: siguen en el SDLC de IOND.** Su versión objetivo se registra en el campo heredado **G.Version**: se llena su valor y no se modifica su configuración.

### Q-45 · Corte de transición para tickets en curso ✅

- **Decisión:** los tickets que aún no llegaron a `dev in progress` adoptan el flujo completo (matriz + VoBo). Los demás terminan con su documentación vigente y siguen las reglas de rechazo, `merged` y Release.

### Q-46 · Convenciones de título y de assignee ✅

- **Decisión:**
  - **Título:** `[PRODUCTO][Módulo] Verbo + objeto`.
  - **Assignee:** quien tiene la siguiente acción. **En** `assignment` **el LD asigna un Dev como assignee.**
  - QA va en Reviewer QA y los revisores de código en Dev Reviewer.

## 2.7 Estrategia de pruebas del release y herramientas de QA (2026-10-10)

### Q-47 · ¿El smoke cambia según la versión? ✅

- **Escenario:** una semana el smoke incluye la facturación porque el release tocó billing y la siguiente no; el resultado del smoke deja de ser comparable entre releases y no se puede automatizar una lista que cambia.

- **Decisión: smoke maestro único y fijo** para cualquier release de iond, IonMind o StageMind: SMK-01 a SMK-17 (login SSO, dashboard, crear y ejecutar un board en test y en producción, historial, listados clave, admin, Admin Gateway y health de los tres productos). Las verificaciones propias de la versión pasan a `sanity`. **El mismo smoke** corre como gate en preprod y después del despliegue en producción.
- **Se aplica en:** fuente canónica §7.2 y §7.7; 05 §6.1; `knowledge/test-suites/smoke-master.md`.

### Q-48 · ¿Qué cubre el sanity? ✅

- **Decisión:** el sanity es **propio de cada versión**: re-ejecuta los REQ críticos de la Matriz de Requerimientos de cada ticket del release (sus TC de la Matriz de Pruebas) en preprod, más las interacciones entre tickets que tocan el mismo módulo o Iond Subcategory. Los IDs siguen el formato `SAN-<ticket>-<REQ>`.
- **Se aplica en:** fuente canónica §7.2; 05 §6.2.

### Q-49 · ¿Cómo se arma la regresión de cada versión? ✅

- **Decisión:** la regresión es **propia de cada versión** y se selecciona del E2E maestro por riesgo (capa 1): baseline crítico fijo, módulos impactados (mapa Iond Subcategory → módulo, con su impacto cruzado), flujos cross-system cuando el release toca integración, contrato de API o autenticación, y la suite extendida cuando el riesgo de la T-04 es Alto.
- **Se aplica en:** fuente canónica §7.2; 05 §6.3.

### Q-50 · ¿Dónde y con qué alcance corre el E2E maestro? ✅

- **Escenario:** el E2E maestro tiene 250 casos y solo una parte está automatizada; con 2 QAs no es posible ejecutarlo completo a mano cada semana.
- **Decisión:**
  - Corre en **cada release, después de la regresión priorizada**, como **capa 2 dentro del estado** `regression` (sin estados nuevos en ClickUp).
  - **La parte automatizada corre completa**, sin supervisión, el jueves por la noche.
  - **Las filas manuales** que la capa 1 no tomó se reparten por módulo en **4 grupos de rotación (G1–G4)**: cada release ejecuta el grupo siguiente, y el maestro completo se cubre cada 4 releases.
  - Una falla de la capa 2 es un hallazgo; solo es NO-GO si es `urgent` y la provocó el release. El resto va a `backlog` con `det-regression`.
- **Se aplica en:** fuente canónica §7.2, §7.4 y §7.7; 05 §6.4 y §9.2.

### Q-51 · ¿Cuál es la fuente del E2E maestro? ✅

- **Decisión:** la hoja **"Iond Testing Plan"** sigue siendo la fuente que edita el equipo. El catalyst asigna **IDs estables** (`E2E-<SIDE>-<MOD>-NNN`) en la primera importación y devuelve las columnas ID, TICKET y SPEC para pegarlas en la hoja; las importaciones siguientes se cruzan por ID y nunca se renumera. Las filas eliminadas quedan como `Retirada`.
- **Se aplica en:** `knowledge/test-suites/`; 05 §6.4.

### Q-52 · ¿El smoke se ejecuta en producción? ✅

- **Escenario:** el smoke crea y ejecuta un board; hacerlo con una cuenta de cliente real contaminaría sus datos, su consumo y sus métricas.
- **Decisión:** sí, con una **company de prueba dedicada** en producción. Los datos que crea el smoke llevan el prefijo `autobot-` y se eliminan al terminar; las verificaciones que no tienen borrado se hacen en modo lectura. Las credenciales se guardan en el `.env` de la máquina del QA, nunca en un repositorio. La company todavía no existe (AV-09).

- **Se aplica en:** fuente canónica §7.3 (ACT-02) y C-31; 05 §6.1.

#### Q-53 · ¿Qué pasa con las subtareas en `done w/d` cuando el padre avanza? ✅

- **Decisión:** cuando el padre pasa a `ready to merge`, sus subtareas en `done w/d` pasan también a `ready to merge`, y lo siguen hasta `merged` (automatización **A-20**; si ClickUp no permite la acción sobre subtareas, el RM las mueve en el C-23, AV-10).
- **Se aplica en:** fuente canónica D-13, D-15, D-16, I-08, I-10, I-11 y A-20; 06a; 06b.

#### Q-54 · ¿Qué alcance tienen las herramientas de QA (catalyst)? ✅

- **Decisión:**
  - El catalyst **lee** ClickUp (solo lectura) y el repositorio de automatización (bot-test) **sin modificarlos**. Genera matrices, comentarios C-xx, comandos de ejecución por suite y la priorización de la automatización; QA y el SM publican.
  - Las skills se organizan por tablero y estado en `skills/` (formato agnóstico) y se exponen a Claude Code con accesos en `.claude/skills/`.
  - Los cambios que necesita bot-test para ejecutar las suites (Ola 0) se registran como requisitos para su responsable (AV-11).
- **Se aplica en:** 01 §4–7; `SKILL.md`; `knowledge/test-suites/automation-roadmap.md`.

### 2.8 Ajustes del flujo (2026-10-11)

#### Q-55 · ¿Cuándo se asigna el QA de un ticket? ✅

- **Escenario:** si el QA se conoce recién en `ready for qa`, quien redactó la matriz puede no ser quien prueba, y el QA llega a `requirements` sin haber seguido el R&D ni el prototipo.
- **Decisión:**
  - En **Discovery**, el **LQA asigna el Reviewer QA en** `assignment`, junto con el Dev y el Dev Reviewer que asigna el LD. El QA sigue el R&D, participa en `prototype review` y llega listo para redactar la matriz en `requirements`.
  - El **mismo QA** ejecuta `mock testing`. En `ready for qa` el LQA solo confirma o reasigna si la capacidad lo exige (C-12).
  - En **Improvements** (sin `assignment`), el LQA asigna al entrar a `requirements` (mejoras con criterios) o en `ready for qa` (bugs y mejoras sin `requirements`).
- **Se aplica en:** fuente canónica §2, D-02, D-03, D-05, D-10, I-02, I-06, §11, C-02, C-12, A-08 y §16; 02; 04 §3 y §8; 05; 06a; 06b; 07.

---

## 3. Acciones de verificación (AV-xx)

| ID | Acción | Responsable | Cuándo | Relacionado |
|---|---|---|---|---|
| AV-01 | Acordar con el SH el **día dedicado** de la Discovery Review semanal y agendarla como evento recurrente | SM + PO | Antes del primer ciclo | Q-07 |
| AV-02 | Inventariar por repositorio el pipeline de CI, la cobertura de pruebas unitarias y el análisis estático | LD | Antes del primer release semanal | L-11, F-22 |

| ID | Acción | Responsable | Cuándo | Relacionado |
|---|---|---|---|---|
| AV-03 | Verificar en el editor de automatizaciones de ClickUp las condiciones por tag y por campo vacío antes de activar cada oleada | SM | Fase F2 | L-05 |
| AV-04 | Configurar ClickUp: crear `blocked` en Discovery; mover `to do` de Discovery a Active después de `requirements`; pasar `done w/d` a Done; ubicar `uat` antes de `ready to merge`; intercambiar `to do` y `requirements` en Improvements; crear el campo **Stagemind.Version** | SM | Fase F2 | Q-41, Q-42 |
| AV-05 | Crear los tags SDLC, las plantillas T-01 a T-04 (con el checklist de acuse) y el Form de intake de bugs | SM | Fase F2 | Q-23, Q-28 |
| AV-06 | Aplicar el Anexo A de la guía 06a (ajustes en tickets existentes) | SM | Fase F2 | Q-44, Q-45 |
| AV-07 | Acordar con DevOps la ventana fija de despliegue y el procedimiento de rollback | LD/RM | Antes del primer release semanal | Q-36 |
| AV-08 | ~~Planificar la matriz de smoke y las features automatizables; priorizar los tickets de automatización; después, sanity y regresión~~ ✅ Resuelta en Q-47 a Q-51 | LQA + QA | Fase F2b | Q-32, Q-40 |
| AV-09 | Crear la **company de prueba en producción**: tenant y usuarios dedicados, aislamiento del billing y de las métricas de negocio, política de limpieza de datos `autobot-`, credenciales solo en el `.env` del QA | LQA + LD/RM | Antes del primer smoke en producción | Q-52 |
| AV-10 | Verificar si las automatizaciones de ClickUp pueden mover subtareas cuando cambia el padre (A-20); si no, documentar el paso manual del RM en el C-23 | SM | Fase F2 | Q-53 |
| AV-11 | Entregar al responsable de bot-test los requisitos de la **Ola 0**: integrar el PR de la rama E2E, corregir los imports de las specs existentes, excluir las specs exploratorias, perfiles de variables por ambiente (preprod y company de prueba) y etiquetas de suite ( `@smoke` , `@prod-safe` ) | LQA | Antes de automatizar el smoke | Q-50, Q-54 |
| AV-12 | Definir el **flujo representativo de StageMind** del smoke (SMK-17) | LQA + LD | Antes del primer release con StageMind | Q-47 |

---

## 4. Observaciones (O-xx)

| ID | Observación | Implicación | Acción |
|---|---|---|---|
| **O-01** | **IOND cuenta con 2 QAs.** Hoy los releases esperan que se completen tickets, por lo que la carga puede cubrirse; **cuando el flujo se estabilice, QA estará al tope**. | 2 QAs = 10 días-QA por semana. El release semanal consume ≈ 3 días-QA si el lote es pequeño. El resto se reparte entre matrices y `mock testing` (ver el cálculo en 05 §9.2). | Límite de lote por QA Points (Q-32), sanity sobre REQ críticos, regresión por riesgo, smoke automatizado (Q-40), revisión en cada retro (R-01). |
| **O-03** | **El RM es el mismo que el LD.** | `assignment` , `code review` , merge a la RC y despliegue al ambiente de QA se concentran en una persona. | Sus PRs requieren la aprobación de dos miembros con mayor antigüedad (Q-02). Documentar un suplente para el corte y el despliegue. |
| **O-04** | **DevOps es un equipo externo.** | El despliegue y su ventana no dependen de IOND. | Ventana fija y rollback acordados (Q-36, AV-07). |
| **O-05** | **Los folders legacy siguen operando** en paralelo al SDLC. | Tags y personas compartidos, con riesgo de doble seguimiento. | Filtrar siempre por las listas SDLC; usar tags SDLC exclusivos (L-04, R-15). |

| ID | Observación | Implicación | Acción |
|---|---|---|---|
| **O-06** | **El release semanal depende de la disciplina del corte.** | Los lotes grandes alargan `sanity` y `regression`. | Corte estricto, release pequeño y frecuente, sin releases vacíos (Q-32). |

---

## 5. Inconsistencias detectadas y su resolución (F-xx)

| ID | Inconsistencia en la guía original | Resolución | Estado |
|---|---|---|---|
| F-01 | `requirements` se describía como "Matriz de Pruebas aprobada con Dev". | Matriz de Requerimientos redactada por QA + VoBo del PO. | ✅ |
| F-02 | `assignment` tenía owner "Dev". | Owner LD, que asigna al Dev como assignee (Q-46). | ✅ |
| F-03 | `prototype review` tenía tres owners y no incluía al LD. | Owner PO; reunión PO + Dev + QA + LD (SM opcional). | ✅ |
| F-04 | `done w/d` tenía varios significados. | Done With Dependencies = cierre de subtareas, tipo Done. | ✅ |
| F-05 | `uat` estaba después de `ready to merge` pero "en staging". | `uat` antes de `ready to merge`, condicional (Q-16). | ✅ |
| F-06 | Improvements tenía UAT en la guía, pero no en el tablero. | Sin UAT en Improvements; excepción decidida por el PO (Q-30). | ✅ |
| F-07 | Improvements tenía el orden `to do` → `requirements`. | `backlog` → `requirements` (opcional) → `to do`. | ✅ |
| F-08 | Discovery no tenía `blocked`. | `blocked` transversal (AV-04). | ✅ |
| F-09 | "Bloqueo > 48 h → mover a Discovery". | Causas `bloq-*` y escalamiento a 48/72 h (Q-31). | ✅ |
| F-10 | Las cajas de tiempo del release sumaban 7,5 días. | Calendario semanal de 5 días (Q-32). | ✅ |
| F-11 | "Regression" tenía significados distintos. | Pruebas de regresión por riesgo + cross-system. | ✅ |
| F-12 | El ejemplo de regresión era de otro equipo. | Flujos de IOND: iond ↔ IonMind ↔ StageMind ↔ Gateway. | ✅ |
| F-13 | DEPLOYMENT figuraba como paso sin estado. | Actividad posterior al Go/No-Go (C-30); los tickets cierran en `merged`. | ✅ |
| F-14 | La validación continua estaba en la guía de Discovery. | Movida a Releases como actividades. | ✅ |
| F-15 | El orden de columnas no seguía el flujo. | `to do` pasa a Active en Discovery; orden documentado (Q-41, AV-04). | ✅ |
| F-16 | `ready to merge`, `uat` y `merged` eran todos Done. | Se mantiene así (Q-41). El tiempo hasta `merged` se mide con Time in Status. | ✅ |
| F-17 | Unanimidad PO + Lead QA + Soporte. | Quórum PO + LQA + RM; Soporte no es un rol del equipo (Q-03, Q-35). | ✅ |
| F-18 | No se definía el destino de un rechazo en Improvements. | Rechazo → `to do`. | ✅ |
| F-19 | No se definía qué hacer con los bugs del release. | Fix-forward para bloqueantes; el resto va a Improvements (Q-34). | ✅ |

| ID | Inconsistencia en la guía original | Resolución | Estado |
|---|---|---|---|
| F-20 | `mock testing` no decía contra qué matriz se valida. | REQ críticos de la Matriz de Requerimientos; para bugs, el resultado esperado del reporte. | ✅ |
| F-21 | La descripción de `merged` en Improvements era una copia de otra. | Corregida. | ✅ |
| F-22 | Se exigían CI, cobertura y análisis estático sin confirmar que existan. | Se mantienen como meta "si el repo lo soporta"; inventario en AV-02. | 🟡 AV-02 |
| F-23 | Pilot sin el Dev y con owner difuso. | SM facilita; el Dev presenta. | ✅ |
| F-24 | No se definía quién despliega en el ambiente de QA. | El LD, con los workflows de DevOps (Q-04). | ✅ |
| F-25 | No había un RM definido. | El RM es el mismo que el LD. | ✅ |
| F-26 | Doble uso de `to do`. | Cola tras el VoBo y retorno de rechazo; bypass de Pilot. | ✅ |
| F-27 | Se proponían custom fields que no pueden crearse. | 13 campos + Stagemind.Version; G.Version para Gateway (Q-42, Q-44). | ✅ |
| F-28 | Varias taxonomías de tags (más de 130 tags). | Taxonomía SDLC única y mapa legacy → SDLC. | ✅ |

---

## 6. Riesgos (R-xx)

**Escala:** Probabilidad (P) e Impacto (I): A = Alta · M = Media · B = Baja.

| ID | Riesgo | P | I | Mitigación |
|---|---|---|---|---|
| R-01 | **Capacidad de QA** (2 QAs) frente al release semanal, las matrices y `mock testing`. | A | A | Límite de lote, sanity sobre REQ críticos, regresión por riesgo, smoke automatizado (Q-32, Q-40), revisión en la retro. |
| R-02 | **Cuello de botella en el VoBo** (un PO para tres productos). | A | A | SLA de 24 h, escalamiento y `blocked` a las 48 h (Q-17), Discovery Review semanal, dashboard de VoBo pendientes. |
| R-03 | **Duplicación** de la especificación entre el PO y QA. | M | M | QA normaliza y completa, no reescribe (Q-19). |
| R-04 | **Autorreferencia:** QA redacta la matriz y juzga contra ella. | M | A | VoBo del PO, checklist de acuse del Dev (Q-23), escalamiento al PO. |
| R-05 | **Requerimientos tardíos** presentados como rechazo. | A | M | Rechazo solo contra la matriz y el contrato; `fuera-de-matriz` (Q-10, Q-22). |
| R-06 | **Funcionalidades incompletas** por ajustes de Pilot. | M | A | Clasificación de ajustes en C-10 (Q-11). |
| R-07 | **Concentración de roles** (SM/QA/LQA en una persona). | A | A | Facilitador alterno LD/RM (Q-03), Go/No-Go colegiado (Q-35), auditoría apoyada en automatizaciones. |
| R-08 | **Concentración LD/RM** en una persona. | A | M | Sus PRs requieren la aprobación de dos miembros senior (Q-02); suplente para el corte y el despliegue. |
| R-09 | **Merge parcial multi-repo** o dependencias entre productos. | M | A | Merge atómico (Q-33), tareas de Release relacionadas y orden de despliegue (Q-37). |
| R-10 | **Release semanal vacío o sobrecargado.** | M | M | Sin releases vacíos y con límite de lote (Q-32). |

| ID | Riesgo | P | I | Mitigación |
|---|---|---|---|---|
| R-11 | **Sobrecarga de reuniones.** | M | M | Discovery Review semanal y SH por video (Q-06, Q-07). |
| R-12 | **Incumplimiento del flujo** en ClickUp (no hay bloqueo nativo). | A | M | Automatizaciones reactivas, auditoría del SM, dashboard de cumplimiento. |
| R-13 | **Dependencia del DevOps externo.** | M | A | Ventana fija y rollback probado (Q-36, AV-07). |
| R-14 | **Salidas no deterministas** de IA. | A | M | Umbrales sobre un dataset y ejecuciones repetidas (Q-24). |
| R-15 | **Doble seguimiento** por los folders legacy. | M | M | Vistas y dashboards filtrados por las listas SDLC; tags exclusivos. |
| R-16 | **Ambiente asignado al QA sin mocks dedicados** (datos que cambian, resultados no reproducibles). | M | M | La sección *Datos de prueba* de la matriz define datos y simulaciones; datos de prueba propios por QA (Q-05). |
| R-17 | **Catalyst desalineado** mientras dura su adecuación. | M | M | Refactor por etapas R0–R5 (doc 01); mientras tanto, uso manual de las plantillas C-xx. |
| R-18 | **Automatización insuficiente para el E2E maestro**: hoy solo una parte de las filas se ejecuta de forma automática, y la rotación manual consume capacidad. | A | M | Rotación G1–G4 dimensionada en ≈ 0,5 día-QA; automatización por olas (smoke → baseline → sanity estable); seguimiento del % ejecutable en el reporte semanal (Q-50). |
| R-19 | **Datos de prueba en producción** que contaminan consumo, facturación o métricas. | M | A | Company de prueba aislada, prefijo `autobot-`, limpieza obligatoria y verificaciones de solo lectura donde no hay borrado (Q-52, AV-09). |

## 7. Limitaciones (L-xx)

| ID | Limitación | Cómo se aborda |
|---|---|---|
| L-01 | ClickUp **no impide** transiciones de forma nativa. | Automatizaciones que revierten y comentan, más la auditoría del SM. |
| L-02 | **Custom fields acotados:** 13 originales + Stagemind.Version; los heredados no se modifican (G.Version solo se llena para Gateway). | Tags SDLC, prioridad nativa, relaciones y checklists. |
| L-03 | ~~StageMind sin campo de versión.~~ | ✅ Resuelta: campo **Stagemind.Version** (Q-42). |
| L-04 | Los tags se **comparten** con los folders legacy activos. | Taxonomía exclusiva para el SDLC y filtros por lista. |
| L-05 | Límite de ejecuciones de automatizaciones (~10.000 al mes en Business) y condiciones por verificar. | Activación por oleadas y verificación previa (AV-03). |
| L-06 | **No hay histórico comparable** para Time in Status ni lead time. Además, las tarjetas nativas cortan en el primer estado Done (Q-41). | Línea base durante el piloto; el tiempo hasta `merged` se mide con Time in Status. |
| L-07 | **No hay estado de despliegue** en ClickUp. | Comentario C-30 + cierre de la tarea de Release. |
| L-08 | **No hay ambiente mock dedicado.** | `mock testing` en el ambiente asignado al QA (Q-05). |
| L-09 | El MCP de ClickUp del catalyst es de **solo lectura**. | El SM aplica la configuración (guía 06a). |
| L-10 | **DevOps externo** controla las ventanas de despliegue. | Ventana fija acordada (AV-07). |

| ID | Limitación | Cómo se aborda |
|---|---|---|
| L-11 | CI, cobertura y análisis estático **no confirmados** en todos los repositorios. | Inventario por repositorio (AV-02). |
| L-12 | El incremento de **Rejection_count** puede no ser automatizable. | Recordatorio automático (A-09) y auditoría. |
| L-13 | Las automatizaciones de ClickUp podrían no actuar sobre subtareas al cambiar el padre. | Verificación (AV-10); paso manual del RM en el C-23. |
| L-14 | El catalyst no modifica bot-test: las etiquetas de suite y los perfiles por ambiente dependen de su responsable. | Selección por IDs de caso ( `-g` ) mientras tanto; requisitos de la Ola 0 (AV-11). |

---

## 8. Registro de decisiones

| Q | Decisión | Fecha |
|---|---|---|
| Q-01 | Un PO para IOND, IonMind y StageMind (misma persona); el solicitante externo participa como SH | 2026-10-09 |
| Q-02 | LD = RM; sus PRs los aprueban dos miembros con mayor antigüedad (ambos) | 2026-10-09 |
| Q-03 | Si el SM es parte de un desacuerdo, facilita el LD/RM; Soporte fuera del equipo | 2026-10-09 |
| Q-04 | El LD despliega en el ambiente de QA con los workflows de DevOps | 2026-10-09 |
| Q-05 | `mock testing` en el ambiente asignado al QA | 2026-10-09 |
| Q-06 | SH presencial; por video si no puede asistir | 2026-10-09 |
| Q-07 | Discovery Review semanal en un día dedicado (a acordar con el SH) | 2026-10-09 |
| Q-08 | R&D con timebox S 1 / M 3 / L 5 días; `prototype review` obligatorio | 2026-10-09 |
| Q-09 | Prototipo según el tipo de ticket | 2026-10-09 |
| Q-10 | El contrato aprobado es línea base si la matriz lo referencia | 2026-10-09 |
| Q-11 | Clasificación de los ajustes de Pilot en C-10 | 2026-10-09 |
| Q-12 | Una matriz en el ticket padre | 2026-10-09 |
| Q-13 | REQ de invariancia + REQ-NF en refactors y POC | 2026-10-09 |
| Q-14 | Spikes como subtareas ( `done w/d` ) | 2026-10-09 |
| Q-15 | El PO clasifica Discovery; QA clasifica bugs con aprobación del PO | 2026-10-09 |
| Q-16 | UAT antes de `ready to merge` ( `uat-requerido` ) | 2026-10-09 |

| Q | Decisión | Fecha |
|---|---|---|
| Q-17 | Sin VoBo tácito; 24 h escala, 48 h `blocked` | 2026-10-09 |
| Q-18 | Cambios de alcance solo vía PO (v+1, C-08) | 2026-10-09 |
| Q-19 | QA normaliza y completa; no reescribe | 2026-10-09 |
| Q-20 | QA propone; el PO decide | 2026-10-09 |
| Q-21 | Reglas indefinidas: matriz + C-05, VoBo en 24 h | 2026-10-09 |
| Q-22 | Hallazgos fuera de matriz → ticket nuevo | 2026-10-09 |
| Q-23 | Acuse del Dev como checklist del ticket | 2026-10-09 |
| Q-24 | Umbrales de IA definidos por el PO; dataset propuesto por QA | 2026-10-09 |
| Q-25 | Escala única de severidad (prioridad nativa) | 2026-10-09 |
| Q-26 | QA Points en la capacidad | 2026-10-09 |
| Q-27 | Separar en otro ticket si hay un actor, regla o módulo nuevo | 2026-10-09 |
| Q-28 | Form de intake + triage semanal | 2026-10-09 |
| Q-29 | Criterios para que una mejora pase por `requirements` | 2026-10-09 |
| Q-30 | Sin UAT en Improvements; excepción decidida por el PO | 2026-10-09 |
| Q-31 | Causas `bloq-*`; escalamiento a 48/72 h | 2026-10-09 |
| Q-32 | Calendario semanal, corte el lunes, sin releases vacíos, límite de lote | 2026-10-09 |
| Q-33 | Merge atómico | 2026-10-09 |
| Q-34 | Fix-forward (≤ 4 h / ≤ 1 día) o `diferido` / NO-GO | 2026-10-09 |
| Q-35 | Go/No-Go PO + LQA + RM; CONDITIONAL GO; tratamiento del NO-GO | 2026-10-09 |
| Q-36 | Ventana fija con DevOps; el PO comunica a los clientes | 2026-10-09 |
| Q-37 | Una tarea de Release por producto; despliegue según dependencias | 2026-10-09 |
| Q-38 | Carril de hotfix (≤ 24 h) | 2026-10-09 |
| Q-39 | Bug bash quincenal; post-despliegue 2 h / 48 h | 2026-10-09 |
| Q-40 | Regresión por riesgo; automatizar smoke → regression → sanity parcial | 2026-10-09 |

| Q | Decisión | Fecha |
|---|---|---|
| Q-41 | Tipos actuales (Done para `uat`, `ready to merge` y `merged`; Closed para `invalid`); `to do` de Discovery a Active | 2026-10-09 |
| Q-42 | Campo Stagemind.Version | 2026-10-09 |
| Q-43 | 5 valores de TYPE | 2026-10-09 |
| Q-44 | Gateway sigue en el SDLC; versión en G.Version | 2026-10-09 |
| Q-45 | Corte de transición | 2026-10-09 |
| Q-46 | Convención de título y assignee; el LD asigna al Dev | 2026-10-09 |
| Q-47 | Smoke maestro único y fijo (SMK-01 a SMK-17), en preprod y producción | 2026-10-10 |
| Q-48 | Sanity propio de cada versión (REQ críticos de sus tickets) | 2026-10-10 |
| Q-49 | Regresión propia de cada versión, seleccionada del E2E maestro por riesgo (capa 1) | 2026-10-10 |
| Q-50 | E2E maestro como capa 2 de `regression`: automatizado completo + rotación G1–G4 | 2026-10-10 |
| Q-51 | Hoja "Iond Testing Plan" como fuente editable; IDs estables asignados por el catalyst | 2026-10-10 |
| Q-52 | Smoke en producción con una company de prueba dedicada y limpieza | 2026-10-10 |
| Q-53 | `done w/d` sigue al padre a `ready to merge` y `merged` (A-20) | 2026-10-10 |
| Q-54 | El catalyst lee ClickUp y bot-test sin modificarlos; skills por estado + accesos en `.claude/skills/` | 2026-10-10 |
| Q-55 | El LQA asigna el Reviewer QA en `assignment` (Discovery); en Improvements, al entrar a `requirements` o a `ready for qa` | 2026-10-11 |